# Demanda, Provisionamento e Desempenho

Aplicação memory-bound | Experimentos locais em Docker | 14 de outubro de 2026

Janelas observadas entre 14:44 e 15:47, horário de Fortaleza (UTC-3). Tempo útil de medição: 48 minutos.

Grupo 15. Parte memory-bound: Victor Vili Xavier Luna. Matrícula: 122110387.

Nas 48 execuções, 12 tiveram falhas HTTP na janela e 12 apresentaram evidência de morte por falta de memória no ponto. Com oito usuários, a vazão válida média foi 0,09 req/s em 500 MiB e 1,67 req/s em 4 GiB. Os resultados permitem comparar capacidade de atendimento e pressão de memória, sem atribuir todo o custo da aplicação exclusivamente à RAM.

## Aplicações desenvolvidas

### Aplicação memory-bound

Desenvolvemos um backend Python 3.10 com FastAPI para consolidar cadastros enviados em arquivos CSV. A rota POST /memory-test recebe vários arquivos em ordem, lê os registros incrementalmente e mantém um dicionário com o registro completo de cada ID. A última ocorrência substitui a anterior. A resposta contém apenas contagens e tempo interno de processamento; o cadastro consolidado não é devolvido.

A necessidade de manter os registros únicos até o final produz pressão de capacidade de memória. Entretanto, parsing de texto, construção de objetos Python, GIL, upload e arquivos temporários também consomem recursos. O endpoint síncrono é executado em threads; seis processos Uvicorn permitem paralelismo entre processos. Cada usuário virtual envia uma requisição por vez, sem espera artificial.

| Dataset | Arquivos | Registros | IDs únicos | Substituições |
|---|---|---|---|---|
| Grande | 4 | 96000 | 88000 | 8000 |

O lote possui 99,15 MiB; cada descrição tem 1024 bytes. Os mesmos quatro arquivos e a mesma ordem são usados em todas as requisições; o manifesto registra tamanhos e hashes SHA-256. O volume de arquivos não equivale diretamente à memória residente dos objetos Python ou ao consumo total do cgroup.

### Monitoramento da aplicação e da infraestrutura

Locust no macOS registra respostas válidas, falhas e latências HTTP. Um monitor dentro do container acompanha o processo mestre Uvicorn e seus descendentes, com psutil e cgroup v2. A cada 0,5 segundo coleta CPU, RSS, memory.current, limite de RAM, swap e contadores OOM. Outro monitor acompanha a árvore de processos do gerador no Mac. RSS pode duplicar páginas compartilhadas; memory.current inclui também cache contabilizado ao container.

# Configuração da VM e Deploy da Aplicação

Estas coletas utilizaram containers Docker no Mac ARM64, apoiados na VM Linux interna do Docker Desktop. Os limites de provisionamento foram aplicados ao container por cgroup, e não como RAM/vCPUs de uma VM VirtualBox. Portanto, este relatório descreve experimentos locais Docker; não relata deploy nem medições realizadas no VDI do professor ou no LCC.

| Componente | Configuração registrada |
|---|---|
| Hospedeiro | macOS ARM64; Apple M5; 16 GiB de RAM; Locust fora do container |
| VM interna Docker Desktop | aarch64; 10 CPUs lógicas; 7,75 GiB de RAM |
| Container | Linux ARM64; Python 3.10; 6 CPUs de quota; 6 workers Uvicorn |
| Memória e swap | 500 / 1024 / 2048 / 4096 MiB; swap desabilitado |
| Rede | Porta dinâmica do hospedeiro 127.0.0.1 para porta 8000 do container |
| Imagem | sha256:4edd2fb6c4fbf450bae1617d6812... |

O Dockerfile instala as dependências da API e psutil, copia os fontes e disponibiliza a porta 8000. O executor cria um container novo para cada ponto, monta seu diretório de resultados, inicia seis workers, valida a API com um CSV mínimo e registra as dependências reais. Os datasets ficam no hospedeiro; são enviados por multipart em fluxo e conferidos por manifesto e SHA-256.

```
docker build -t memory-bound:experiment memory-bound
memory-bound/.venv/bin/python memory-bound/scripts/docker_experiment.py
```

O código e protocolo utilizados foram preservados em snapshot, junto de ID da imagem, versão Docker, estado Git, manifesto, configurações efetivas e logs. Ao final de cada ponto, o executor encerra monitores e remove apenas o container que criou. A identificação completa da imagem e os detalhes de CPU/RAM estão em environment.json e container.json.

# Teste de Stress da Aplicação selecionada

## Escolha de CPU e workers

Antes dos testes de RAM, foram comparadas configurações de 4 CPUs/4 workers e 6 CPUs/6 workers, com 4 GiB, oito usuários, o mesmo dataset e três execuções de 30 segundos por configuração. A vazão média foi 1,62 e 1,36 req/s, respectivamente, sem demonstrar ganho ao ampliar CPU/workers. Adotaram-se 6 CPUs/6 workers para ampliar a margem em relação à quota global de CPU, mantendo-os constantes nos 48 experimentos. A calibração teve duração e metodologia próprias; seus resultados não foram misturados aos testes finais. A escolha não garante gargalo exclusivo de memória nem permite concluir que seis CPUs causem pior desempenho.

| Cenário | RAM | CPU | Workers | Demandas | Repetições |
|---|---|---|---|---|---|
| C1 | 500 MiB | 6 | 6 | 1, 2, 4, 8 | 3 por demanda |
| C2 | 1024 MiB | 6 | 6 | 1, 2, 4, 8 | 3 por demanda |
| C3 | 2048 MiB | 6 | 6 | 1, 2, 4, 8 | 3 por demanda |
| C4 | 4096 MiB | 6 | 6 | 1, 2, 4, 8 | 3 por demanda |

A matriz possui 4 configurações de RAM × 4 demandas × 3 repetições, totalizando 48 execuções. Cada ponto tem rampa de N usuários a N usuários/s, aquecimento de 10 segundos após todos estarem ativos e janela de medição de 60 segundos. Há 5 segundos entre containers. A ordem rotaciona demanda e RAM e inverte RAM na segunda rodada para reduzir efeitos da ordem. Aquecimento não garante estado estacionário; o histórico temporal permanece disponível.

## Metodologia de comparação

Vazão válida é o número de respostas corretas concluídas dentro da janela dividido por 60 segundos. Conclusões de requisições iniciadas no aquecimento contribuem para vazão, mas não para latência. Latência média e p95 principais incluem somente sucessos iniciados e concluídos na janela. Falhas são apresentadas separadamente; sem sucesso, a latência é indisponível, não zero.

Requisições iniciadas na janela e ainda pendentes no final são censuradas e contadas explicitamente. Elas não são classificadas artificialmente como falha ou sucesso. Médias e p95 podem favorecer respostas curtas quando há censura, por isso devem ser lidos junto de pendentes e vazão. O p95 usa nearest rank. Os CSVs acumulados do Locust incluem aquecimento; a comparação usa load/measurement.json e os registros individuais de requests.csv.

No modelo fechado sem espera, é esperado haver requisições pendentes em qualquer instante. Seu número isolado não demonstra falha nem saturação; ele informa a censura da amostra.

Cada ponto é uma observação independente para agregação. As tabelas e gráficos mostram média e desvio padrão amostral entre as três repetições; p95 é a média dos p95 de cada repetição, não um percentil de todas as requisições misturadas. Três repetições não estabelecem significância estatística nem intervalos de confiança confiáveis.

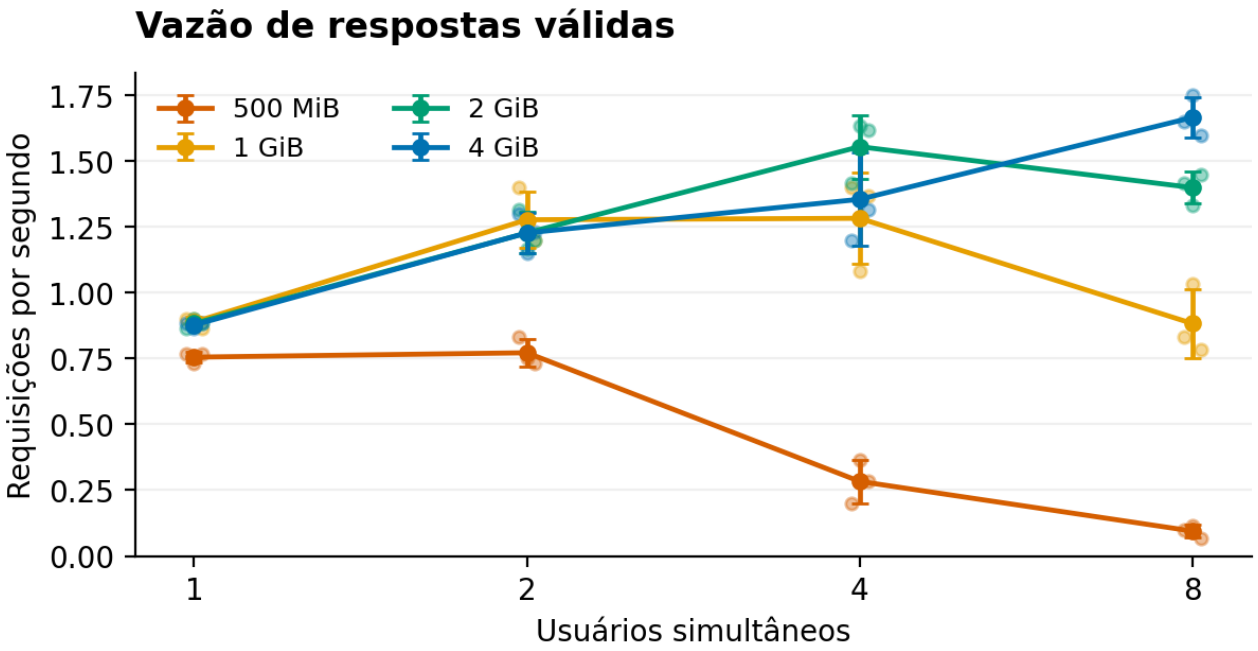
A auditoria conferiu 768 condições e não encontrou divergências entre limites reais, registros individuais e métricas derivadas. 0 pontos não produziram janela comparável. 9 execuções tiveram cobertura de recursos incompleta. Essas amostras são preservadas, mas não entram nas médias comparativas de CPU/memória; as tabelas informam o número de observações disponíveis.

## Resultados comparativos de desempenho

| RAM MiB | Usuários | n HTTP/lat | Vazão válida req/s | Latência média s | p95 s | Falhas % |
|---|---|---|---|---|---|---|
| 500 | 1 | 3/3 | 0,76 ± 0,02 | 1,33 ± 0,03 | 1,68 ± 0,06 | 0,0 ± 0,0 |
| 500 | 2 | 3/3 | 0,77 ± 0,05 | 1,85 ± 0,15 | 2,52 ± 0,20 | 30,2 ± 0,6 |
| 500 | 4 | 3/3 | 0,28 ± 0,08 | 3,09 ± 0,24 | 4,61 ± 0,85 | 78,8 ± 5,3 |
| 500 | 8 | 3/3 | 0,09 ± 0,03 | 5,83 ± 0,61 | 8,61 ± 1,10 | 94,4 ± 1,2 |
| 1024 | 1 | 3/3 | 0,89 ± 0,02 | 1,13 ± 0,02 | 1,18 ± 0,04 | 0,0 ± 0,0 |
| 1024 | 2 | 3/3 | 1,28 ± 0,11 | 1,56 ± 0,14 | 1,92 ± 0,15 | 0,0 ± 0,0 |
| 1024 | 4 | 3/3 | 1,28 ± 0,17 | 3,11 ± 0,49 | 4,41 ± 1,09 | 0,0 ± 0,0 |
| 1024 | 8 | 3/3 | 0,88 ± 0,13 | 5,00 ± 0,30 | 8,08 ± 0,87 | 40,2 ± 4,7 |
| 2048 | 1 | 3/3 | 0,88 ± 0,02 | 1,13 ± 0,02 | 1,18 ± 0,04 | 0,0 ± 0,0 |
| 2048 | 2 | 3/3 | 1,23 ± 0,08 | 1,62 ± 0,10 | 1,73 ± 0,13 | 0,0 ± 0,0 |
| 2048 | 4 | 3/3 | 1,56 ± 0,12 | 2,55 ± 0,23 | 3,31 ± 0,48 | 0,0 ± 0,0 |
| 2048 | 8 | 3/3 | 1,40 ± 0,06 | 5,71 ± 0,10 | 11,75 ± 0,42 | 0,0 ± 0,0 |
| 4096 | 1 | 3/3 | 0,88 ± 0,01 | 1,14 ± 0,02 | 1,20 ± 0,04 | 0,0 ± 0,0 |
| 4096 | 2 | 3/3 | 1,23 ± 0,08 | 1,62 ± 0,09 | 1,74 ± 0,06 | 0,0 ± 0,0 |
| 4096 | 4 | 3/3 | 1,36 ± 0,18 | 2,96 ± 0,37 | 4,47 ± 0,65 | 0,0 ± 0,0 |
| 4096 | 8 | 3/3 | 1,67 ± 0,08 | 4,78 ± 0,10 | 6,84 ± 1,36 | 0,0 ± 0,0 |

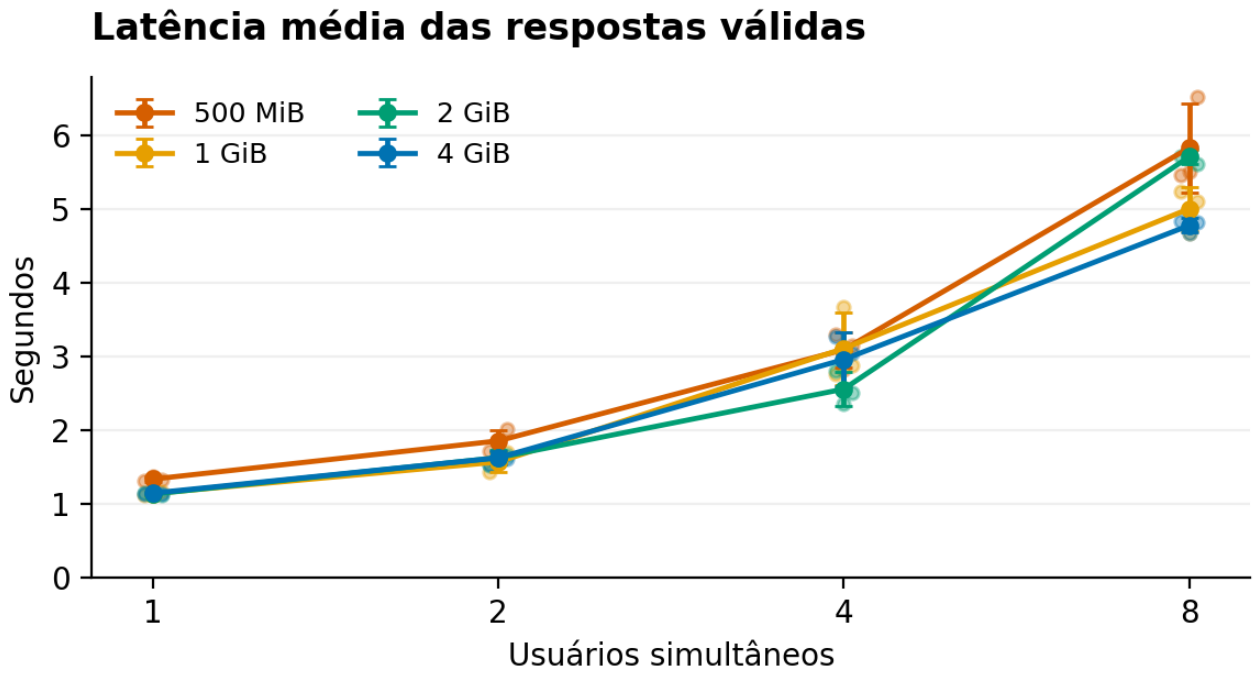
Valores = média ± desvio padrão entre repetições. n HTTP/lat = janelas completas / repetições com latência de sucesso disponível. Traço indica ausência de observação.

## Demanda e desempenho




Média ± desvio padrão entre repetições; pontos claros = execuções individuais.

Figura 1. Vazão válida por demanda e provisionamento. Pontos claros mostram cada repetição.

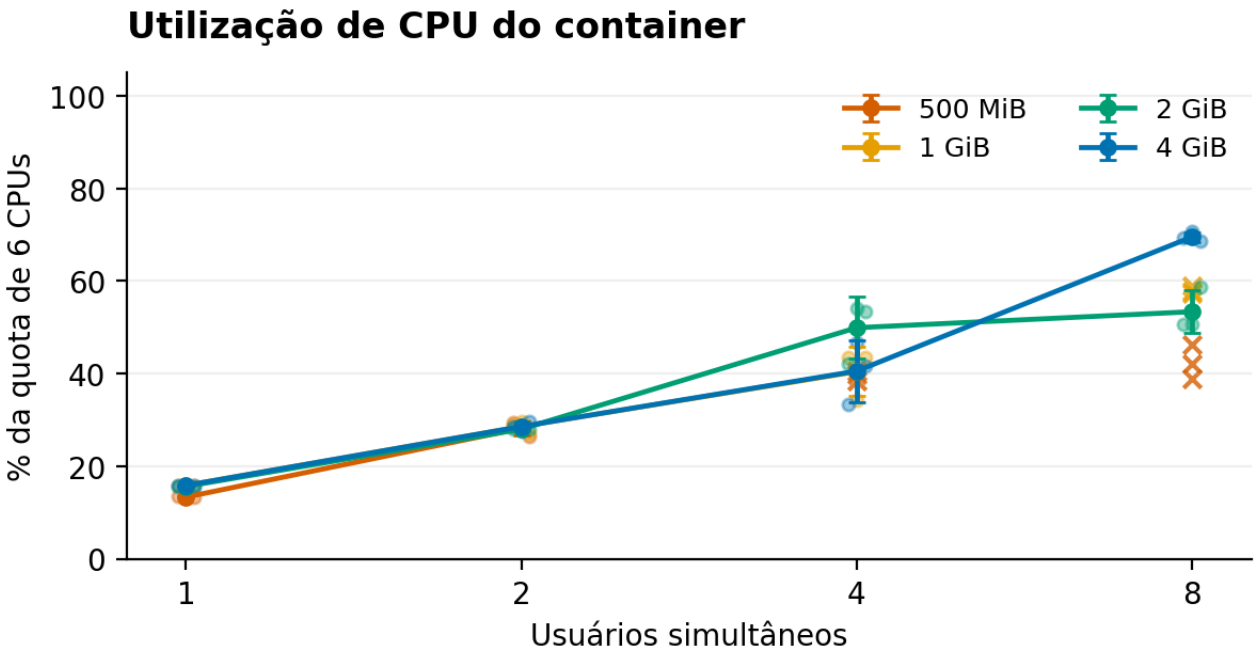



Média ± desvio padrão entre repetições; pontos claros = execuções individuais.

Figura 2. Latência média dos sucessos inteiramente contidos na janela; ler junto de pendentes e falhas.

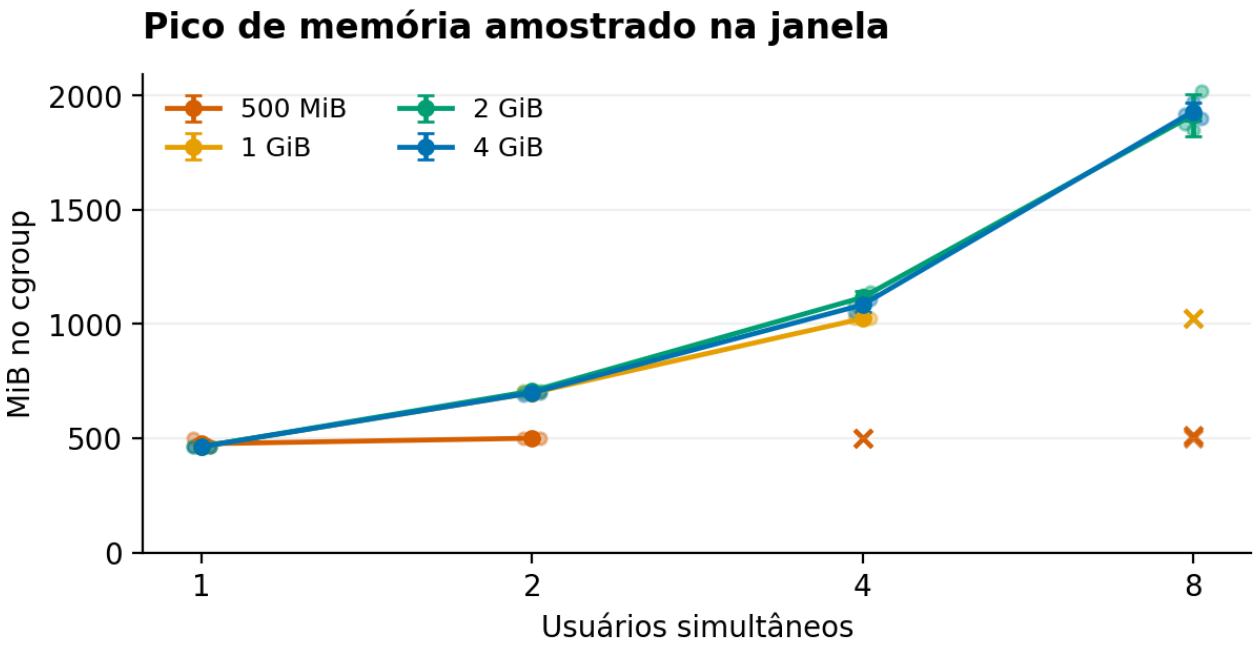
Em 4 GiB, aumentar a demanda de um para oito usuários alterou a vazão média de 0,88 para 1,67 req/s e a latência média das respostas válidas de 1141 para 4777 ms. A memória máxima amostrada média passou de 463,7 para 1929,9 MiB. Mais usuários representam mais requisições simultâneas em um modelo fechado, não uma taxa fixa de chegada.

## Utilização dos recursos




Média ± desvio padrão; × = coleta com lacunas, fora da média.

Figura 3. CPU normalizada pela quota de seis CPUs. Cruzes indicam coletas com lacunas, fora das médias.

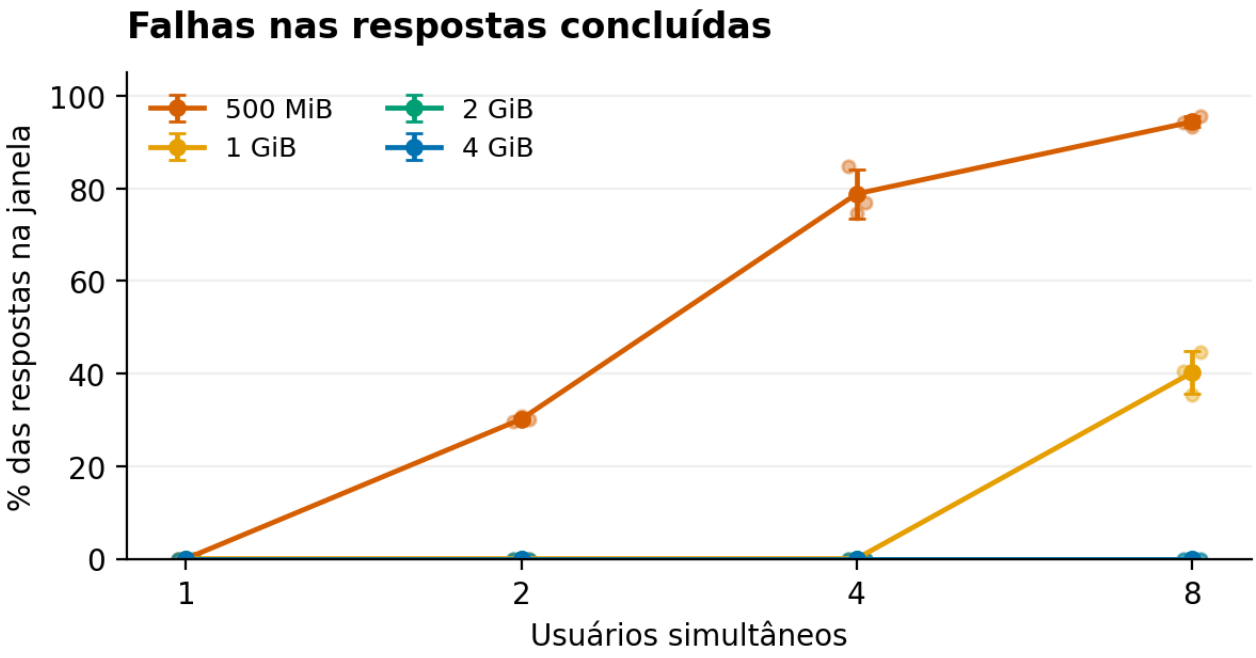



Média ± desvio padrão; × = coleta com lacunas, fora da média.

Figura 4. Pico amostrado do cgroup. Cruzes indicam coletas parciais; picos podem ter sido perdidos nas lacunas.

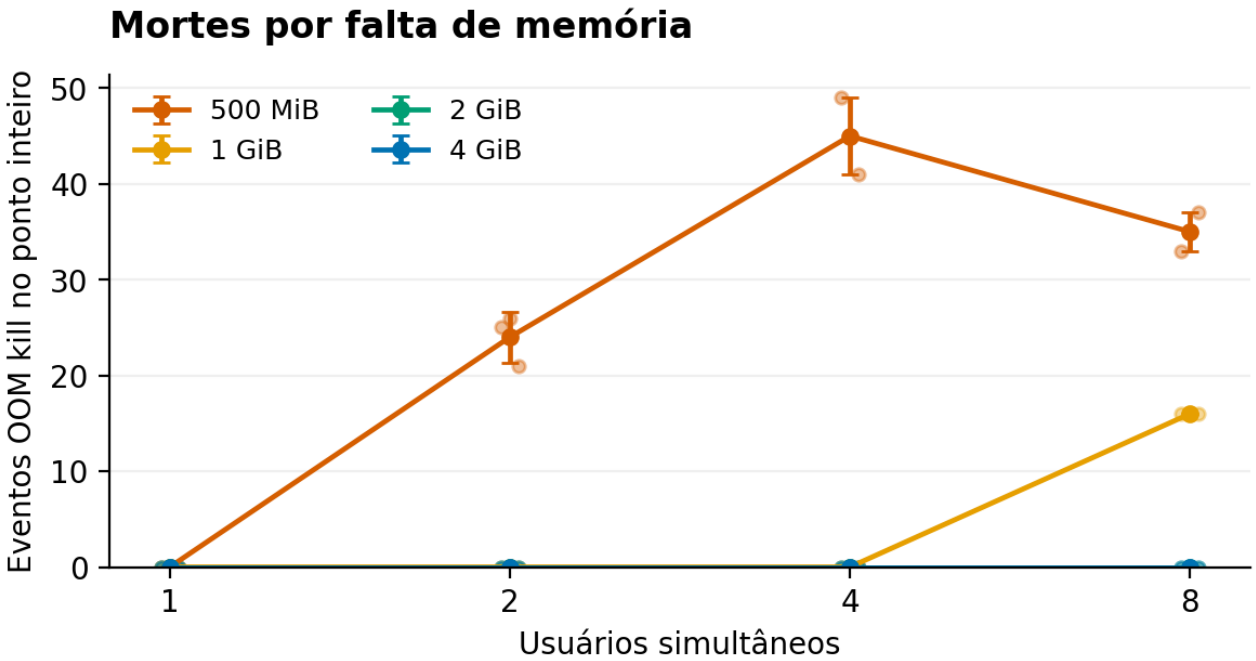
CPU é calculada pela diferença do contador cumulativo entre amostras dentro da janela, dividida pelo intervalo real e pela quota. O pico de memória é amostrado e pode perder picos entre leituras. Métricas de sistema do psutil refletem a VM interna Docker, não o limite de RAM do container.

## Falhas e falta de memória




Média ± desvio padrão entre repetições; pontos claros = execuções individuais.

Figura 5. Percentual de falhas entre respostas concluídas na janela.

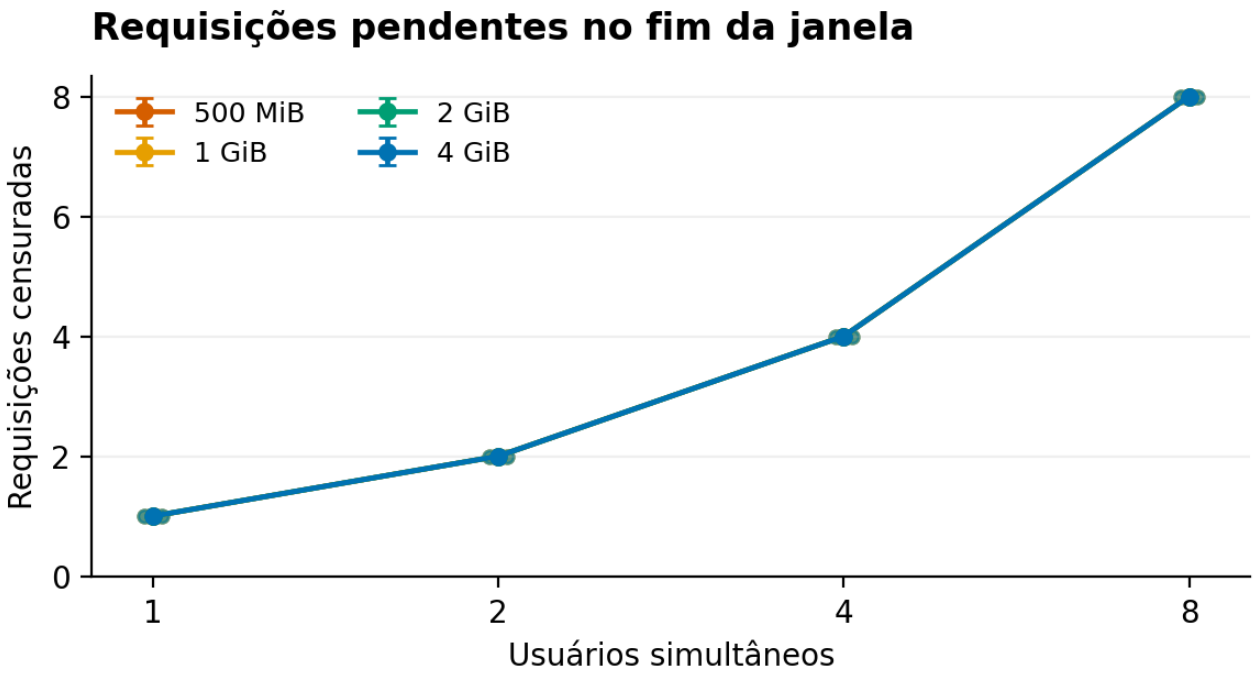



Média ± desvio padrão entre repetições; pontos claros = execuções individuais.

Figura 6. Eventos OOM kill contabilizados no ponto inteiro, incluindo aquecimento.

Na demanda de oito usuários, a proporção média de falhas foi 94,4% com 500 MiB e 0,0% com 4 GiB. Ler esses percentuais junto da vazão válida e dos eventos OOM evita interpretar respostas rápidas de erro como bom desempenho. Os eventos de OOM abrangem aquecimento e medição; o total do ponto não é automaticamente um total só da janela.

### Requisições pendentes e interpretação




Média ± desvio padrão entre repetições; pontos claros = execuções individuais.

Figura 7. Requisições ainda em andamento no fim da janela. Não equivalem a falhas.

500 MiB: 0,09 ± 0,03 req/s, 5,83 ± 0,61 s e 3/3 pontos com OOM

1024 MiB: 0,88 ± 0,13 req/s, 5,00 ± 0,30 s e 3/3 pontos com OOM

2048 MiB: 1,40 ± 0,06 req/s, 5,71 ± 0,10 s e 0/3 pontos com OOM

4096 MiB: 1,67 ± 0,08 req/s, 4,78 ± 0,10 s e 0/3 pontos com OOM

500 MiB: primeiro nível com OOM em 2 usuários; 1024 MiB: primeiro nível com OOM em 8 usuários; 2048 MiB: OOM não observado até oito usuários; 4096 MiB: OOM não observado até oito usuários. Esses são os primeiros níveis observados na grade testada, não limiares universais.

O efeito de provisionamento deve ser avaliado para a mesma demanda. OOM indica pressão de capacidade real. Por outro lado, ampliar RAM além do conjunto necessário não garante aumento da vazão: parsing, upload, armazenamento temporário, GIL e distribuição das conexões entre workers continuam participando do tempo total.

## Conclusões

Nas 48 execuções, 12 tiveram falhas HTTP na janela e 12 apresentaram evidência de morte por falta de memória no ponto. Com oito usuários, a vazão válida média foi 0,09 req/s em 500 MiB e 1,67 req/s em 4 GiB. Os resultados permitem comparar capacidade de atendimento e pressão de memória, sem atribuir todo o custo da aplicação exclusivamente à RAM.

As conclusões valem para este backend, dataset, hospedeiro e modelo fechado de carga. Não se mediram temperatura, frequência, pressão global de memória do macOS ou desempenho físico do armazenamento. O computador não foi reservado exclusivamente ao experimento. Não extrapolar os limiares observados para outro hardware nem afirmar gargalo exclusivamente de RAM. A etapa VirtualBox/LCC, se mantida na avaliação, exige medição própria.

## Detalhamento de recursos e rastreabilidade

| RAM MiB | Usuários | n recursos | CPU % quota | Pico RAM MiB | Pontos OOM/3 | Pendentes |
|---|---|---|---|---|---|---|
| 500 | 1 | 3 | 13,3 ± 0,1 | 475,6 ± 20,8 | 0/3 | 1,0 ± 0,0 |
| 500 | 2 | 3 | 28,2 ± 1,5 | 500,1 ± 0,2 | 3/3 | 2,0 ± 0,0 |
| 500 | 4 | 0 | - | - | 3/3 | 4,0 ± 0,0 |
| 500 | 8 | 0 | - | - | 3/3 | 8,0 ± 0,0 |
| 1024 | 1 | 3 | 15,8 ± 0,1 | 464,0 ± 1,8 | 0/3 | 1,0 ± 0,0 |
| 1024 | 2 | 3 | 28,6 ± 1,3 | 698,6 ± 9,7 | 0/3 | 2,0 ± 0,0 |
| 1024 | 4 | 3 | 40,4 ± 5,3 | 1024,0 ± 0,0 | 0/3 | 4,0 ± 0,0 |
| 1024 | 8 | 0 | - | - | 3/3 | 8,0 ± 0,0 |
| 2048 | 1 | 3 | 15,6 ± 0,3 | 463,3 ± 0,5 | 0/3 | 1,0 ± 0,0 |
| 2048 | 2 | 3 | 28,0 ± 0,1 | 706,8 ± 3,7 | 0/3 | 2,0 ± 0,0 |
| 2048 | 4 | 3 | 49,9 ± 6,7 | 1116,7 ± 28,8 | 0/3 | 4,0 ± 0,0 |
| 2048 | 8 | 3 | 53,4 ± 4,7 | 1914,8 ± 90,7 | 0/3 | 8,0 ± 0,0 |
| 4096 | 1 | 3 | 15,8 ± 0,1 | 463,7 ± 0,7 | 0/3 | 1,0 ± 0,0 |
| 4096 | 2 | 3 | 28,6 ± 1,0 | 698,8 ± 9,7 | 0/3 | 2,0 ± 0,0 |
| 4096 | 4 | 3 | 40,5 ± 6,7 | 1084,2 ± 30,0 | 0/3 | 4,0 ± 0,0 |
| 4096 | 8 | 3 | 69,5 ± 1,0 | 1929,9 ± 37,8 | 0/3 | 8,0 ± 0,0 |

n recursos conta janelas com cobertura adequada. Contadores finais OOM abrangem todo o ponto; quando o container encerra antes da leitura, o estado OOMKilled permanece evidência, mas a quantidade de mortes pode ficar indisponível. Não preencher ausência com zero.

Diretório dos dados brutos: docker-experiments-20261014T174435803277Z.

Arquivos derivados: output/analysis/runs.csv, comparison.csv, audit.json e gráficos PNG/SVG. Cada ponto preserva configuração, validation.json, measurement.json, requests.csv, CSVs dos monitores, logs, dependências e docker inspect. O relatório é reproduzido pelos scripts analyze_results.py e build_report.py.

Os dados brutos ficam ignorados pelo Git durante a coleta. O snapshot e o manifesto permitem conferir exatamente o código e os CSVs usados; incluir os artefatos selecionados na entrega do grupo. O template fornecido orientou as três seções principais; esta parte cobre apenas memory-bound.